

TRADER APP

Spécifications Fonctionnelles & Techniques

Version 1.1 • Mars 2026

Projet	Trader App — Application mobile pour traders
Version	V1.1 — mise à jour architecture & stack
Marchés	Tous marchés — selon le choix de l'utilisateur
Plateforme	Mobile iOS & Android (Expo + React Native)
IA	Aucune en V1 — zéro coût API

1. Contexte & Objectifs

Trader App est une application mobile destinée aux traders travaillant sur compte personnel, tous marchés confondus (forex, crypto, indices, matières premières, actions). L'objectif reste de centraliser les outils essentiels du quotidien du trader dans une seule application : gestion du plan de trading, suivi des news de marché et bilan hebdomadaire guidé, sans recours à l'intelligence artificielle en V1.

Problèmes résolus par l'application :

- Absence d'un espace centralisé pour consulter et éditer son plan de trading
- News de marché trop génériques, non filtrées selon les paires tradées
- Aucun outil de bilan hebdomadaire structuré et guidé sur mobile
- Difficulté à obtenir rapidement la tendance HTF d'une paire avant d'analyser

2. Périmètre Fonctionnel V1

2.1 Authentification

- Création de compte via email + mot de passe
- Connexion email + mot de passe
- Gestion de session via JWT (Spring Security)
- Déconnexion et invalidation du token

2.2 Plan de trading

Le plan de trading reste le cœur de l'application. Il alimente automatiquement le filtrage des news, certaines préférences de notification et la vision globale du trader.

- Saisie manuelle via formulaire structuré à champs standards imposés
- Déclaration de champs custom avant téléchargement du template
- Téléchargement du template au format DOCX, PDF ou Notes
- Génération dynamique du template selon les champs standards + custom
- Upload d'un plan structuré puis mapping dans les champs connus
- Édition de chaque champ en un tap
- Commentaires par champ (idées en attente, notes personnelles)
- Export du plan et partage direct via WhatsApp / Discord

Champs standards du plan

- Paires tradées
- Sessions de trading (London, New York, Asia...)
- Stratégie / setup utilisé
- Règles d'entrée en position

- Règles de sortie de position
- Risque par trade (%)
- Risk/Reward minimum
- Ce qu'il faut éviter
- Objectifs

Modélisation des champs custom

Table	Rôle	Champs clés
trading_plan	Stockage des champs standards fixes du plan	id, user_id, paires, sessions, strategie, regles...
trading_plan_custom_field	Stockage des champs personnalisés ajoutés par le trader	id, trading_plan_id logique, field_name, field_value, display_order

Exemple de génération dynamique

```
TradingPlanTemplate template = new TemplateBuilder()
    .addStandardFields()
    .addCustomField("Killzones")
    .build(Format.DOCX);
```

Les adaptateurs de sortie retenus sont : Apache POI pour DOCX, iText/PDFBox pour PDF et un générateur texte pour le format Notes.

2.3 News & Marché

- Tendance HTF par paire depuis le plan — calcul via MA200 + structure highs/lows
- News filtrées automatiquement selon les paires du plan de trading
- News d'autres paires choisies manuellement par l'utilisateur
- Notifications push par paire — activables et désactivables individuellement
- Calendrier économique filtré — événements à impact fort uniquement
- Headlines filtrées par mots-clés des devises composant les paires tradées
- Projection locale des préférences nécessaires au module News

2.4 Bilan hebdomadaire

- Bot conversationnel scripté (machine à états) — sans IA
- Déclenchement chaque dimanche
- 5 à 6 questions guidées (nombre de trades, winrate, respect du plan...)
- Génération automatique d'un résumé à partir des réponses
- Historique des bilans consultable

2.5 Notifications

- Envoi de notifications push via Firebase Cloud Messaging (FCM)
- Envoi d'emails transactionnels via JavaMailSender + Resend (SMTP)
- Notifications in-app via cloche et flux SSE
- Préférences configurables par paire et par type d'événement
- Point d'entrée unique pour les canaux push, email et in-app

3. Hors Scope V1

Feature	Raison
Fear & Greed Index	Uniquement crypto — pas universel pour tous les traders
Circuit breaker	Dépend trop de la bonne volonté de l'utilisateur
Calculateur de position	Déjà disponible sur des applications dédiées
Analyse IA par screenshot	Trop complexe pour la V1 et nécessite une API vision
Format libre du plan	Remplacé par une structure imposée pour fiabilité du mapping
Checklist pré-trade	Reportée en V2

WebSocket temps réel	Les besoins V1 sont couverts par FCM + SSE
API Gateway	Non retenue à ce stade : complexité inutile pour un monolithe modulaire

4. Architecture Technique

4.1 Vue d'ensemble

L'application repose sur un monolithe modulaire Spring Boot. Le front mobile Expo communique directement avec le backend via Axios. En développement, la base URL de l'API est injectée via variables d'environnement Expo. Plus tard, un reverse proxy Nginx pourra être placé devant le backend pour gérer SSL, domaine, headers et rate limiting. Aucune API Gateway n'est prévue à ce stade.

Flux de communication V1 :

- Expo (Axios + variables d'environnement) → Spring Boot modulaire
- Spring Boot → PostgreSQL + RabbitMQ + FCM + Resend
- PDF.js / traitement front → envoi de données structurées au backend
- SSE → remontée des notifications in-app vers la cloche côté mobile

4.2 Stack technique

Couche	Technologie
Mobile	Expo + React Native
Bundler mobile	Metro (via Expo)
HTTP client	Axios
Cache requêtes serveur	TanStack Query (React Query)
État global	Zustand
Stockage local	React Native Async Storage
Fichiers mobiles	Expo Document Picker, Expo File System, Expo Sharing
Notifications mobile	Expo Notifications + FCM
Backend	Spring Boot 4.0.4 — monolithe modulaire
Java	Java 21
Build backend	Gradle
Base de données	PostgreSQL + Flyway
Messaging	RabbitMQ
Lecture / génération de documents	PDF.js, Apache POI, iText/PDFBox
Documentation API	Swagger / OpenAPI
Containerisation	Docker + Docker Compose

Tendance des paires	TwelveData API ou Alpha Vantage
News & calendrier	NewsAPI + ForexFactory API

4.3 Modules du monolithe & schémas PostgreSQL

Le monolithe est découpé en cinq modules cohérents. Chaque module dispose de son propre schéma PostgreSQL. Aucune foreign key inter-module n'est autorisée. Les échanges de données utiles se font par événements RabbitMQ, puis chaque module maintient sa projection locale.

Module	Schéma	Tables / données principales
Auth	auth	users, tokens
Plan	plan	trading_plan, trading_plan_custom_field
News	news	news_preferences (copie locale)
Notifications	notifications	notification_prefs, logs
Bilan	bilan	bilan_session, bilan_reponse

4.4 Architecture hexagonale par module

Chaque module respecte l'architecture domain-application-infrastructure. Le service métier est placé dans la couche application et orchestre les ports sans dépendre des détails techniques.

```

plan/
├── domain/
│   ├── TradingPlan.java
│   └── TradingPlanRepository.java
├── application/
│   ├── ports/in/UpdatePlanUseCase.java
│   ├── ports/out/SavePlanPort.java
│   ├── services/TradingPlanService.java
│   └── usecases/UpdatePlanUseCaseImpl.java
└── infrastructure/
    ├── persistence/TradingPlanJpaRepository.java
    ├── web/TradingPlanController.java
    ├── messaging/RabbitMqEventPublisher.java
    └── template/DocxTemplateExportAdapter.java

```

4.5 CQRS & projections locales

Module	Hexagonale	CQRS	Justification
Auth	Oui	Non	CRUD simple

Plan	Oui	Oui	Lecture fréquente vs écriture rare
News	Oui	Oui	Principalement de la lecture + projections locales
Bilan	Oui	Non	Flux question / réponse simple
Notifications	Oui	Non	Commandes d'envoi + préférences

Le pattern retenu est l'Event-Carried State Transfer : chaque module reçoit via RabbitMQ les données dont il a besoin et maintient sa propre copie locale. Exemple : le module News conserve news_preferences comme projection des paires suivies depuis le plan.

4.6 Patterns de conception identifiés

Pattern	Application dans Trader App
Builder	Génération dynamique des templates de plan (DOCX, PDF, Notes)
Strategy	Choix du canal de notification selon le contexte
Factory	Création des événements RabbitMQ et de leurs payloads
Template Method	Orchestration du bot de bilan hebdomadaire
Decorator	Enrichissement progressif des news avant affichage
Observer (remplacé)	Remplacé par une communication asynchrone via RabbitMQ

4.7 Middleware & sécurité

- JwtAuthFilter — vérification du token JWT sur chaque requête protégée
- LoggingFilter — traçage des requêtes
- RateLimitingFilter — limitation du trafic côté backend
- CorsFilter — autorisation des appels depuis Expo
- GlobalExceptionHandler — réponses d'erreur cohérentes

4.8 Docker & environnement local

L'ensemble de la stack locale est containerisé via Docker Compose. En développement, le mobile appelle directement l'URL du backend fournie par variable d'environnement Expo. Nginx n'est pas requis localement dans la V1.

services:

backend: # Spring Boot monolithe modulaire

db: # PostgreSQL

rabbitmq: # Broker d'événements

mailhog: # Tests email locaux optionnels

5. CI/CD de démarrage

Une chaîne GitHub Actions est mise en place dès le début du projet, non pas pour couvrir un métier encore inexistant, mais pour stabiliser le socle technique et l'industrialisation.

- Checkout du dépôt
- Installation de Java 21 et de Node.js
- Cache Gradle / npm ou pnpm
- Compilation / build du backend
- Type-check et lint du front mobile
- Vérification de build des applications
- Contrôles qualité simples sur chaque push et pull request

Remarque : les tests unitaires ne sont pas inclus dans ce pipeline initial. Ils seront ajoutés progressivement lorsque la logique métier existera réellement et apportera une valeur de validation fonctionnelle.

6. APIs & Services Externes

Service	Usage	Coût
TwelveData / Alpha Vantage	Données de prix pour le calcul de tendance HTF	Gratuit (plan free)
NewsAPI / équivalent	Headlines filtrées par paires	Gratuit (plan free)
ForexFactory API	Calendrier économique	Gratuit
Firebase Cloud Messaging	Notifications push iOS & Android	Gratuit
Resend / SMTP	Emails transactionnels	Gratuit jusqu'à un certain volume
PDF.js	Extraction / prétraitement front du plan	Open source — gratuit
Apache POI / iText / PDFBox	Génération de templates et exports	Open source ou usage compatible projet
RabbitMQ	Transport des événements internes	Open source

7. Évolutions prévues V2

- Intégration IA pour analyse de marché personnalisée selon le plan
- Checklist pré-trade générée automatiquement depuis les champs du plan
- Journal de trades — suivi P&L, tags de stratégie, screenshots
- Approfondissement du moteur de notifications
- Ajout éventuel d'un reverse proxy Nginx en production pour SSL, headers et rate limiting
- Migration vers microservices si la charge le justifie réellement

8. Glossaire

Terme	Définition
HTF	Higher Time Frame — timeframe supérieur utilisé pour déterminer la tendance générale
MA200	Moyenne mobile sur 200 périodes — indicateur de tendance long terme
FCM	Firebase Cloud Messaging — service Google pour notifications push mobile
CQRS	Command Query Responsibility Segregation — séparation lectures / écritures
JWT	JSON Web Token — standard d'authentification sans état côté serveur
Port (hexagonale)	Interface définissant un contrat entre la logique métier et l'extérieur
Adaptateur	Implémentation concrète d'un port (BDD, REST, messaging, template...)
Event-Carried State Transfer	Transmission d'un état métier par événement pour maintenir des projections locales
Monolithe modulaire	Application unique découpée en modules indépendants et cohésifs